

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

09 - Árvores rubro-negras, árvores B e tries

Prof. Alex Kutzke

Setembro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Enunciar as propriedades de uma árvore rubro-negra e verificar se uma árvore colorida dada as satisfaz;
2. Explicar por que essas propriedades limitam a altura a $2 \log_2(n + 1)$;
3. Descrever a inserção em termos de recoloração e rotação, e dizer o que cada conserto corrige;
4. Comparar AVL e rubro-negra quanto à altura obtida e ao número de consertos por operação, e escolher entre as duas a partir do perfil de uso;
5. Explicar por que a árvore binária é a estrutura errada em disco, e como a árvore B resolve o problema;
6. Executar à mão a inserção com divisão de nó em uma árvore B;
7. Calcular quantos níveis uma árvore B precisa dado o número de chaves por nó;
8. Descrever a trie e justificar por que o custo de busca nela não depende do número de chaves;
9. Escolher entre ABB balanceada, árvore B e trie a partir do tipo de chave e do meio em que a estrutura vive.

Este conteúdo cai na Prova 2.

Roteiro

Rubro-negras

Inserção

AVL e rubro-negra

Árvores B

Tries

Fechamento

Demonstração: o índice de um banco de dados

```
CREATE TABLE aluno(grr INTEGER, nome TEXT);
WITH RECURSIVE seq(n) AS (
    SELECT 1 UNION ALL SELECT n + 1 FROM seq
    WHERE n < 2000000
)
INSERT INTO aluno(grr, nome)
    SELECT n, 'aluno ' || n FROM seq;

EXPLAIN QUERY PLAN
    SELECT nome FROM aluno WHERE grr = 1999999;

CREATE INDEX idx_grr ON aluno(grr);

EXPLAIN QUERY PLAN
    SELECT nome FROM aluno WHERE grr = 1999999;
```

Dois milhões de linhas, a mesma consulta duas vezes.

O plano de execução muda

QUERY PLAN

--SCAN aluno

aluno 1999999

Run Time: real 0.069

QUERY PLAN

--SEARCH aluno USING INDEX idx_grr (grr=?)

aluno 1999999

Run Time: real 0.000

4096 <- PRAGMA page_size

De 69 ms para menos de 1 ms. O índice é uma **árvore B**, e 4096 é o tamanho em bytes de cada nó dela. Voltamos a esta tela no fim da aula.

Três estruturas, a partir da ABB balanceada

Árvore rubro-negra

Mantém a garantia de $O(\log n)$ com um critério de balanceamento mais frouxo que o da AVL. É a árvore de busca das bibliotecas de linguagem.

Árvore B

Abandona o formato binário para funcionar em disco. É a estrutura de índice de bancos de dados e de sistemas de arquivos.

Trie

Abandona a comparação entre chaves inteiras e usa a estrutura interna da chave, letra por letra.

O que o critério da AVL cobra

Alturas das duas subárvores diferindo em no máximo 1, em todo nó.

- um campo de altura por nó, atualizado em todo nó do caminho;
- a remoção pode rotacionar em cada nível até a raiz: até $O(\log n)$ rotações para tirar **uma** chave.

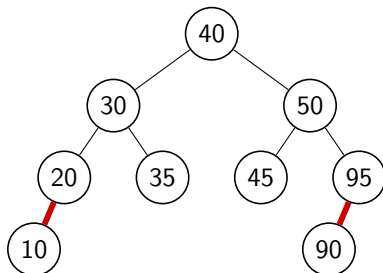
A inserção é barata (no máximo uma rotação simples ou dupla).
A remoção é que decide em uma estrutura que recebe alterações o tempo todo.

Pergunta: quanto desbalanceamento dá para admitir sem perder a altura logarítmica?

As três propriedades

Uma ABB em que cada nó tem uma cor e vale:

1. a raiz é preta;
2. nenhum nó vermelho tem filho vermelho;
3. todo caminho da raiz até uma subárvore vazia tem o mesmo número de nós pretos.



Esse número é a **altura preta**. Aqui, 3.

Desenho de Sedgewick: a cor do nó é a da **ligação** que vem do pai. Vermelhos: 10 e 90.

Por que a altura é logarítmica

Seja b a altura preta.

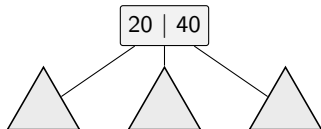
- o caminho mais curto só tem pretos: b nós;
- o mais longo alterna preto e vermelho (propriedade 2): no máximo $2b$ nós.

Nenhum caminho passa do dobro de outro. Com pelo menos $2^b - 1$ nós na árvore, vale $b \leq \log_2(n + 1)$ e

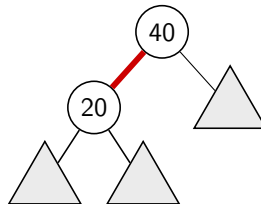
$$h \leq 2 \log_2(n + 1)$$

Contra $1,44 \log_2(n + 2)$ da AVL. O limite é pior; em troca, na versão do Cormen, a remoção faz menos rotações que na AVL.

As cores codificam uma árvore 2-3



Nó 2-3 com duas chaves e três filhos.



A ligação **vermelha** une as duas chaves do mesmo nó 2-3.

- ligação preta = descer um nível na árvore 2-3;
- altura preta = altura da árvore 2-3.

A variante left-leaning

Sedgewick (seção 3.3) acrescenta duas exigências às três propriedades:

- toda ligação vermelha aponta para a **esquerda**;
- nenhum nó tem duas ligações vermelhas.

Com elas, cada nó 2-3 tem uma única representação binária, e a correspondência com a árvore 2-3 é exata. A inserção perde metade dos casos e cabe em três testes.

Duas versões de rubro-negra

Cormen (cap. 13)

- vermelho à direita permitido, e nó preto com os dois filhos vermelhos (um nó de três chaves);
- as cores codificam uma árvore **2-3-4**;
- no máximo 2 rotações por inserção e 3 por remoção;
- é a versão das bibliotecas.

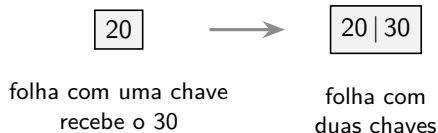
Left-leaning (Sedgewick, 3.3)

- vermelho só à esquerda, e nenhum nó com duas ligações vermelhas;
- as cores codificam uma árvore **2-3**;
- até $O(\log n)$ rotações por inserção e por remoção, em troca de um código bem mais curto;
- é a versão desta aula.

As duas satisfazem as três propriedades e têm o mesmo limite de altura.

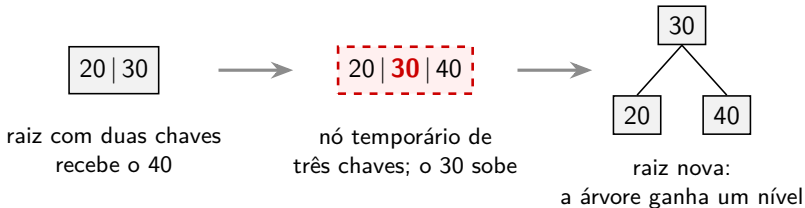
Inserção na árvore 2-3

- a chave nova entra sempre em uma **folha**;
- exemplo: 20, 30, 40, 50, 60, 70 e 80, nessa ordem, a sequência que degenera a ABB em uma lista.



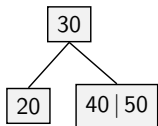
Folha com uma chave: a nova entra ao lado dela, e acabou.

Folha com duas chaves: o nó temporário

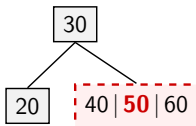


- por um momento, o nó fica com **três chaves** e quatro filhos: um nó que a árvore 2-3 não admite e a **2-3-4** admite;
- ele se divide: a chave do meio sobe, e as outras duas ficam em dois nós de uma chave;
- se o nó é a raiz, a chave do meio vira uma raiz nova.

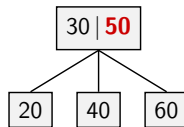
A chave que sobe entra no pai



a folha com duas
chaves recebe o 60



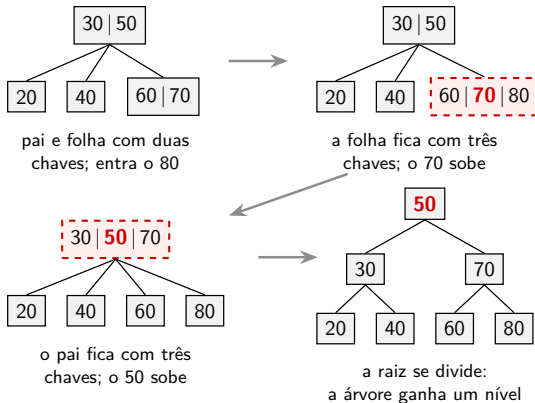
a folha fica com três
chaves; o 50 sobe



o pai tinha uma chave
e fica com duas

Depois de inserir o 70, que só completa a folha do 60, a árvore tem pai e folha com duas chaves.

A divisão pode subir até a raiz



- a árvore cresce pela raiz, e todas as folhas ficam no mesmo nível;
- na rubro-negra, depois das rotações, o nó temporário é o nó com duas ligações **vermelhas** para os filhos, e a divisão é a inversão de cores.

O nó novo entra vermelho

Se entrasse preto

Acrescentaria um nó preto ao caminho que o contém e quebraria a propriedade 3, a mais cara de restabelecer.

Entrando vermelho

Só pode quebrar a propriedade 2, e apenas quando o pai também for vermelho.

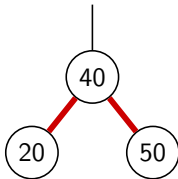
Os consertos acontecem na volta da recursão, em cada nó do caminho da folha nova até a raiz. Ao final, a raiz é pintada de preto.

Os três consertos

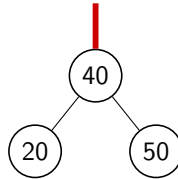
Situação no nó	Conserto	Muda
filho direito vermelho e esquerdo preto	rotação à esquerda	ponteiros
filho esquerdo vermelho com filho esquerdo vermelho	rotação à direita	ponteiros
os dois filhos vermelhos	inversão de cores	só as cores

Nessa ordem, sempre. A inversão de cores empurra o vermelho um nível para cima: na árvore 2-3, é a divisão de um nó que ficou com três chaves.

Inversão de cores



antes



depois

Nenhum ponteiro muda. As duas ligações vermelhas de baixo viram pretas, e a ligação que chega ao 40 vira vermelha: o vermelho sobe um nível para ser tratado pelo pai. A altura preta dos caminhos não muda.

Rotação com cor

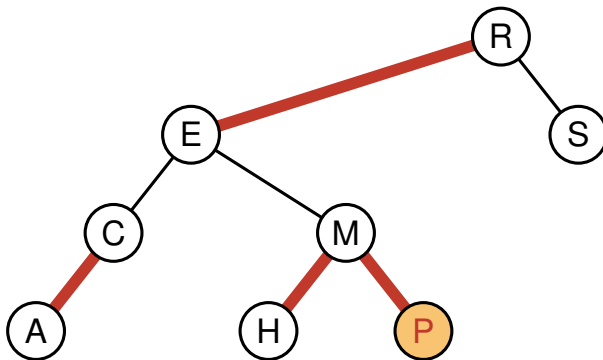
```
func rotacionaEsquerda(a *No) *No {  
    b := a.Dir  
    a.Dir = b.Esq  
    b.Esq = a  
    b.Cor = a.Cor  
    a.Cor = Vermelho  
    return b  
}
```

As três primeiras linhas são a rotação da AVL. As duas seguintes são o que a cor acrescenta: quem sobe leva a cor do nó que estava ali, e quem desce fica vermelho.

A inserção inteira

```
func insere(a *No, v int) *No {
    if a == nil {
        return &No{Info: v, Cor: Vermelho}
    }
    if v < a.Info {
        a.Esq = insere(a.Esq, v)
    } else if v > a.Info {
        a.Dir = insere(a.Dir, v)
    }
    if vermelho(a.Dir) && !vermelho(a.Esq) {
        a = rotacionaEsquerda(a)
    }
    if vermelho(a.Esq) && vermelho(a.Esq.Esq) {
        a = rotacionaDireita(a)
    }
    if vermelho(a.Esq) && vermelho(a.Dir) {
        inverteCores(a)
    }
    return a
}
```

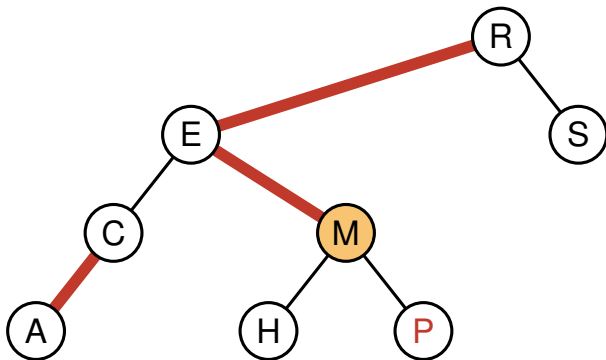
Inserção de P: os consertos em sequência



1. P entra como folha vermelha.

Exemplo de Sedgwick e Wayne, seção 3.3. Em laranja, o nó em que o conserto foi aplicado.

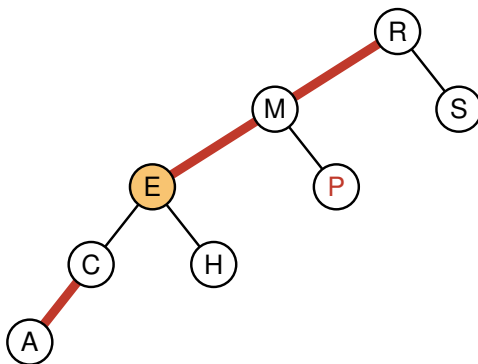
Inserção de P: os consertos em sequência



2. Dois filhos vermelhos: inversão de cores em M.

Exemplo de Sedgwick e Wayne, seção 3.3. Em laranja, o nó em que o conserto foi aplicado.

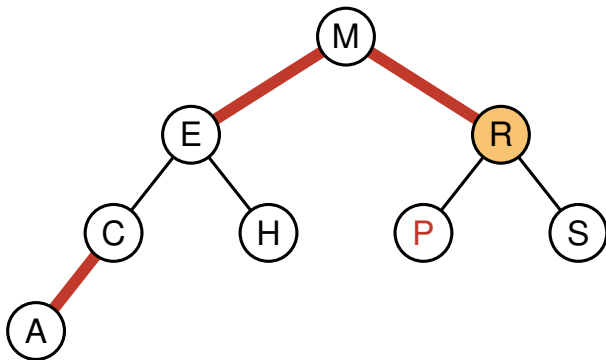
Inserção de P: os consertos em sequência



3. Filho direito vermelho: rotação à esquerda em E.

Exemplo de Sedgwick e Wayne, seção 3.3. Em laranja, o nó em que o conserto foi aplicado.

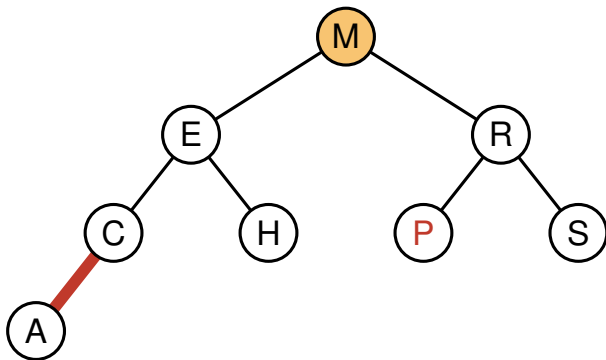
Inserção de P: os consertos em sequência



4. Dois vermelhos seguidos à esquerda: rotação à direita em R.

Exemplo de Sedgewick e Wayne, seção 3.3. Em laranja, o nó em que o conserto foi aplicado.

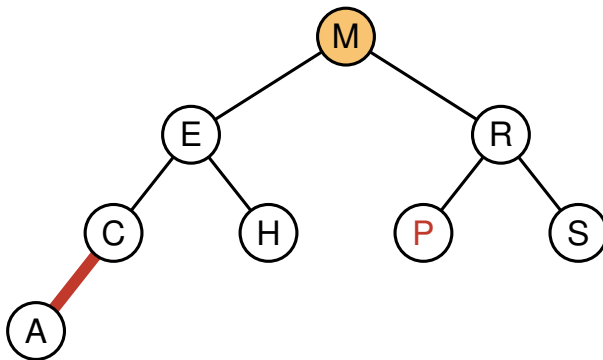
Inserção de P: os consertos em sequência



5. Dois filhos vermelhos: inversão de cores em M.

Exemplo de Sedgwick e Wayne, seção 3.3. Em laranja, o nó em que o conserto foi aplicado.

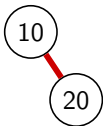
Inserção de P: os consertos em sequência



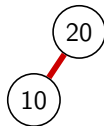
6. A raiz volta a ser preta.

Exemplo de Sedgwick e Wayne, seção 3.3. Em laranja, o nó em que o conserto foi aplicado.

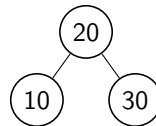
Inserindo 10, 20 e 30 em ordem



vermelho à direita



rotação à esquerda



insere 30, inversão de cores

A entrada em ordem crescente, que degenera a ABB em uma lista, produz aqui uma árvore cheia. O 20 virou raiz sem ter sido inserido primeiro.

A árvore de exemplo, agora colorida

```
$ go run rn.go
```

```
Inserindo [50 30 90 20 40 95 10 35 45]
```

```
      95 (p)
        90 (v)
    50 (p)
      45 (p)
40 (p)
  35 (p)
 30 (p)
  20 (p)
    10 (v)
```

```
altura: 3   altura preta: 3   é rubro-negra: true
```

(v): ligação vermelha vinda do pai. É a árvore do slide das propriedades, e a raiz é o 40, e não o 50 que entrou primeiro.

Medindo as duas: altura e consertos

Inserções em ordem crescente

n	alt. ABB	alt. AVL	alt. RN	rot. AVL	rot. RN	recolor. RN
1000	999	9	9	990	991	985
10000	9999	13	13	9986	9987	9982
100000	99999	16	16	99983	99984	99978

Inserções em ordem aleatória

n	alt. ABB	alt. AVL	alt. RN	rot. AVL	rot. RN	recolor. RN
1000	19	11	13	718	1204	747
10000	31	15	17	7027	11970	7489
100000	40	19	22	69951	118478	74636

go run comparacao.go

O que a tabela diz

- as duas eliminam a degeneração, e ficam bem abaixo dos limites teóricos (24 e 34 para $n = 100$ mil);
- a AVL fica 2 ou 3 níveis mais baixa na ordem aleatória. Cada nível é uma comparação a mais por busca;
- na inserção, a rubro-negra **não** faz menos rotações. Na variante left-leaning medida aqui faz mais, porque também mantém as ligações vermelhas todas à esquerda, e ainda recolore em cerca de três quartos das inserções.

A vantagem da rubro-negra aparece na remoção, que a tabela não mede, e só na versão do Cormen.

A diferença está na remoção

	AVL	RN (Cormen)	RN (left-leaning)
Altura no pior caso	$1,44 \log_2(n + 2)$	$2 \log_2(n + 1)$	$2 \log_2(n + 1)$
Rotações por inserção	no máximo 2	no máximo 2	até $O(\log n)$
Rotações por remoção	até $O(\log n)$	no máximo 3	até $O(\log n)$
Informação por nó	altura (um inteiro) ou fator (2 bits)	cor (1 bit)	cor (1 bit)
Uso típico	muitas buscas, poucas alterações	uso geral, bibliotecas	ensino, implementações curtas

Na AVL, o conserto da remoção pode reduzir a altura da subárvore e propagar o desbalanceamento para o pai, nível a nível. A left-leaning também pode rotacionar em cada nível do caminho.

Muitas remoções: rubro-negra do Cormen. Quase só buscas: AVL.

Onde a rubro-negra é usada

- `std::map` e `std::set` em C++, nas implementações usuais;
- `TreeMap` e `TreeSet` em Java;
- o kernel do Linux, em `lib/rbtree.c`, usado por vários subsistemas.

Todas usam a versão do Cormen.

O `map` de Go fica de fora: ele é uma tabela de dispersão, assunto da próxima aula. Em Go, uma estrutura ordenada exige biblioteca externa ou implementação própria.

A remoção na rubro-negra não é desenvolvida nesta aula e não cai na prova além do limite de 3 rotações na versão do Cormen. Cormen, seção 13.4.

Por que as bibliotecas escolhem a rubro-negra

Mudanças na estrutura em número fixo

Na versão do Cormen, no máximo 2 rotações por inserção e 3 por remoção. Pesa quando a rotação é cara: árvore aumentada (recalcular o valor guardado no nó), estrutura persistente (copiar o nó), acesso concorrente (sincronizar).

A biblioteca não conhece o uso

Pior caso aceitável na busca e na alteração, sem saber a proporção entre as duas.

Tradição

STL da HP/SGI, Cormen, e as bibliotecas que vieram depois.

A diferença prática é pequena: com muitas buscas, a AVL costuma ganhar (o kernel do Windows usa AVL). A memória também não decide: a cor cabe em 1 bit, que o Linux guarda no ponteiro para o pai, e o fator da AVL cabe em 2. A LLRB medida rotaciona mais que a versão do Cormen, das bibliotecas.

Quando a árvore binária é a estrutura errada

Índice com 100 milhões de chaves, em disco.

- árvore rubro-negra: altura por volta de 40;
- cada nível visitado está em um endereço qualquer do arquivo;
- disco rotacional: alguns milissegundos por leitura, mais de 100 ms no total. SSD: dezenas de microssegundos, contra nanossegundos da memória.

E o sistema operacional não lê 16 bytes: lê uma **página** inteira, de 4096 bytes. Ler 16 e ler 4096 custa quase o mesmo.

A árvore binária usa 16 dos 4096 bytes que a leitura trouxe.

O nó do tamanho da página

Chave mais ponteiro ocupando 20 bytes, página de 4096 bytes:

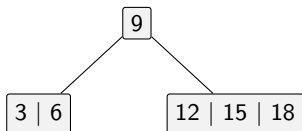
cerca de **200 chaves** e **201 filhos** por nó.

ordem	chaves/nó	1 mil	1 mi	1 bilhão
5	4	5	9	13
101	100	2	3	5
201	200	2	3	4
1001	1000	1	2	3

Níveis com os nós cheios, ou seja, leituras de disco por busca. Um bilhão de chaves em 4 leituras.

Árvore B de ordem m

- cada nó guarda no máximo $m - 1$ chaves em ordem crescente, e no máximo m filhos;
- as chaves do nó dividem as das subárvores em faixas;
- todo nó, exceto a raiz, guarda ao menos $\lceil m/2 \rceil - 1$ chaves;
- **todas as folhas estão no mesmo nível.**



O critério é mais forte que o da AVL: não há caminho mais longo que outro.

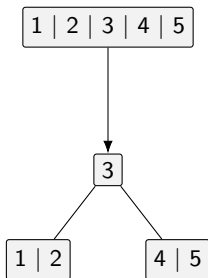
Busca

```
func Busca(n *NoB, v int) bool {  
    if n == nil {  
        return false  
    }  
    i, achou := posicao(n, v)  
    if achou {  
        return true  
    }  
    if ehFolha(n) {  
        return false  
    }  
    return Busca(n.Filhos[i], v)  
}
```

Dentro do nó, a busca escolhe entre m caminhos, e não entre 2. Sequencial ou binária dá no mesmo: as duas acontecem dentro da página já lida.

Inserção: sempre em folha, com divisão

Ordem 5, folha cheia com [1 2 4 5], chega o 3:



Cinco chaves em um nó que só comporta quatro. A chave do meio **sobe** para o pai, e as outras ficam em dois nós.

A divisão em código

```
func divide(n *NoB) (int, *NoB) {
    meio := len(n.Chaves) / 2
    subiu := n.Chaves[meio]

    direito := &NoB{Chaves: append([]int{},
        n.Chaves[meio+1:]...)}
    if !ehFolha(n) {
        direito.Filhos = append([]*NoB{},
            n.Filhos[meio+1:]...)
        n.Filhos = n.Filhos[:meio+1]
    }
    n.Chaves = n.Chaves[:meio]

    return subiu, direito
}
```

A árvore cresce pela raiz

A chave que sobe pode estourar o pai, que se divide também. A divisão pode propagar até a raiz.

Quando a raiz se divide

A árvore ganha um nível. É o único jeito de uma árvore B crescer em altura, e é por isso que todas as folhas continuam no mesmo nível.

Uma árvore binária cresce pelas folhas, e por isso um ramo pode ficar mais longo que os outros.

20 chaves em ordem crescente, ordem 5

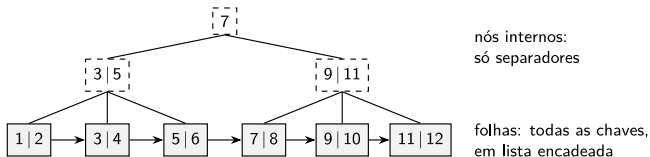
```
$ go run btree.go
inserção do 5: a raiz se dividiu, altura agora é 1
inserção do 17: a raiz se dividiu, altura agora é 2
```

```
[9]
  [3 6]
    [1 2]
    [4 5]
    [7 8]
  [12 15 18]
    [10 11]
    [13 14]
    [16 17]
    [19 20]
```

A mesma entrada que degenera a ABB (altura 19) dá 3 níveis aqui.

B+ e o que os sistemas reais usam

Na **árvore B+**, as chaves ficam todas nas folhas, os nós internos guardam apenas separadores, e as folhas são ligadas em lista encadeada.



As duas mudanças servem à consulta por faixa: achada a chave inicial, o resto do intervalo sai seguindo os ponteiros entre folhas.

- SQLite: tabelas são árvores B+, índices são árvores B;
- PostgreSQL e MySQL/InnoDB: índices em árvore B+;
- sistemas de arquivos como Btrfs e XFS.

De volta à demonstração

```
--SCAN aluno                                69 ms
--SEARCH aluno USING INDEX idx_grr          < 1 ms
--SEARCH ... USING COVERING INDEX idx_grr
      (grr>? AND grr<?)
```

- SCAN: lê todas as páginas da tabela, $\Theta(n)$;
- SEARCH: desce a árvore B, 3 níveis para 2 milhões de chaves;
- o BETWEEN também usa o índice, porque as chaves estão em ordem;
- os 0,43 s do CREATE INDEX são 2 milhões de inserções em árvore B. Indexar acelera a consulta e custa na escrita.

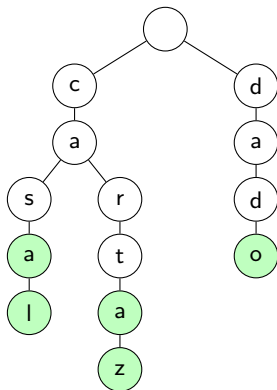
Chaves que têm estrutura interna

Até aqui, toda estrutura compara chaves inteiras. Com chaves de texto, cada comparação percorre a palavra até a primeira letra diferente, e a árvore de altura $\log n$ faz $\log n$ dessas comparações.

A **trie** (de **retrieval**) usa uma letra da chave por nível para escolher o filho.

A chave não fica guardada em nó nenhum: ela é o **caminho** da raiz até o nó, e um marcador diz quais caminhos são palavras inteiras.

Uma trie com cinco palavras



casa, casal, carta,
cartaz, dado.

Nó **verde** marca fim de
palavra.

13 nós além da raiz, para
24 letras. Os prefixos
comuns são guardados
uma vez só.

Nó e inserção

```
type No struct {
    Filhos      map[rune]*No
    FimDePalavra bool
}

func Insere(raiz *No, palavra string) {
    atual := raiz
    for _, letra := range palavra {
        proximo, existe := atual.Filhos[letra]
        if !existe {
            proximo = NovoNo()
            atual.Filhos[letra] = proximo
        }
        atual = proximo
    }
    atual.FimDePalavra = true
}
```

O custo não depende do número de chaves

Operação	Trie	ABB balanceada
Busca de palavra de m letras	$O(m)$	$O(m \log n)$
Consulta por prefixo	$O(m + k)$	percorre a faixa
Espaço	um nó por letra	um nó por chave

Mil ou dez milhões de palavras: a busca custa o mesmo, porque o caminho percorrido é a própria palavra.

Em troca, a trie ocupa mais espaço. O nó pode ser um `map` (pouco espaço, poucos filhos) ou um vetor com uma posição por letra do alfabeto (acesso constante, posições desperdiçadas).

Consulta por prefixo

```
$ go run trie.go
```

```
17 palavras, 86 letras no total, 37 nós na trie
```

```
Contem("cas") = false, EhPrefixo("cas") = true
```

```
ComPrefixo("car") = carro, carta, cartaz, cartão
```

```
ComPrefixo("cas") = casa, casaco, casal, caso, cassino
```

```
ComPrefixo("z") = nenhuma palavra
```

Desce até o nó do prefixo e coleta a subárvore. Nenhuma palavra fora da resposta é visitada.

Autocompletar, corretor ortográfico e roteamento IP por prefixo mais longo. Tabela de dispersão não faz isso: o hash destrói a vizinhança entre chaves parecidas.

Escolher entre as estruturas

Estrutura	Busca	Escolha quando
ABB sem balanceamento	$O(h)$	nunca em código de produção
AVL	$\Theta(\log n)$	busca domina, alterações raras
Rubro-negra	$\Theta(\log n)$	uso geral, muitas alterações
Árvore B / B+	$\Theta(\log_m n)$	a estrutura vive em disco
Trie	$O(m)$	chaves são cadeias, consulta por prefixo

Nas três primeiras linhas, o critério é a proporção entre buscas e alterações. Na quarta, o meio em que a estrutura vive. Na quinta, o tipo da chave.

Para a próxima aula

- exercícios 1 a 12 do roteiro da aula, sem entrega e sem nota. O conteúdo cai na Prova 2;
- rodar `rn.go`, `comparacao.go`, `btree.go` e `trie.go`, alterando as sequências de inserção;
- leitura: Sedgwick, seções 3.3 e 5.2; Cormen, capítulos 13 e 18.

Próxima aula: tabelas de dispersão. A estrutura que abre mão da ordem entre as chaves para buscar em tempo constante.